1. Server-side Design Doc

`uno-sp` does not use a network server. The “server-side” should be treated as the game engine/model layer: rules, deck, players, turn order, card effects, UNO penalties, and game state.

Purpose

The server-side/game-engine layer owns all UNO rules and state. It decides whose turn it is, which cards are playable, what happens when a card is played, when players draw cards, when UNO penalties apply, and when the game ends.

Main Responsibilities

- Create and shuffle a full UNO deck.
- Deal 7 cards to each player.
- Maintain draw pile and discard pile.
- Track current player.
- Track play direction.
- Track UNO calls.
- Validate whether a card is playable.
- Apply action-card effects.
- Detect winner.
- Expose state to the client/rendering layer.

Core Classes

`Card`

- Abstract base class for all UNO cards.
- Fields:
  - `CardColor color`
  - `Image image`
  - `boolean playable`
  - `float angle`
  - `Shape bounds`
- Main methods:
  - `isCompatibleWith(Card topCard)`
  - `render(Graphics g)`
  - `rotate(float degree)`
  - `isClicked(Point2D point)`

`NumberCard`

- Represents cards `0-9`.
- Has `int value`.

- Compatible if:
  - same color as top card, or
  - same number as top number card.

`ActionCard`

- Represents `Skip`, `Reverse`, `+2`, `Wild`, `+4`.
- Has `CardSymbol symbol`.
- Compatible if:
  - black card, or
  - same color, or
  - same action symbol.
- Wild and +4 can change color after being played.

`DrawPile`

- Builds the UNO deck.
- Deck contents:
  - For each color red/yellow/green/blue:
    - one `0`
    - two of each `1-9`
    - two `Skip`
    - two `Reverse`
    - two `+2`
  - Four `Wild`
  - Four `+4`
- Shuffles deck.
- Provides first discard card, ensuring it is not an action card.

`DiscardPile`

- Stores played cards.
- Top card determines play compatibility.
- Starts with first valid card from draw pile.

`Player`

- Abstract base class for human and bot players.
- Fields:
  - `Hand hand`
  - `String name`
  - `DrawPile drawPile`
  - `DiscardPile discardPile`
  - `SeatPosition position`
  - `int id`
  - `Card playedCard`
- Responsibilities:
  - draw cards

  - count cards
  - count playable cards
  - render hand
  - execute a turn

`HumanPlayer`

- Waits for mouse click input from the client side.
- Plays clicked playable card.
- If a number card is selected, also plays matching number cards.
- If wild/+4 is played, asks user to choose color.

`Bot`

- Chooses a random playable card.
- If wild/+4 is played, chooses a random color.

`UnoGame`

- Main game-engine controller.
- Fields:
  - `DrawPile drawPile`
  - `DiscardPile discardPile`
  - `Player[] players`
  - `int currentPlayerIndex`
  - `int direction`
  - `Player currentPlayer`
  - `boolean[] unoCalledByPlayer`
- Responsibilities:
  - initialize game
  - run turn loop
  - apply Skip, Reverse, +2, +4
  - check UNO call
  - apply UNO penalty
  - detect winner
  - transition to game-over state

Turn Flow

1. Set `currentPlayer`.
2. Reset that player's UNO-call flag.
3. If an action effect is pending:
   - `Skip`: skip current player.
   - `+2`: current player draws 2 and skips.
   - `+4`: current player draws 4 and skips.
4. Otherwise current player takes turn.
5. Check if current player has 0 cards.
6. If yes, game ends.

7. If current player has 1 card:
   - if UNO was called, continue.
   - otherwise draw 2 penalty cards.
8. If played card is `Reverse`, flip direction.
9. If played card is `Skip`, `+2`, or `+4`, mark action pending.
10. Advance to next player.

UNO Rule

The current player may call UNO only when they have exactly 2 cards before playing down to 1.

If a player reaches 1 card without calling UNO, they draw 2 penalty cards.

Game End

The game ends when any player has 0 cards. The winner is stored as `UnoGame.currentPlayer`, then the app enters `GameOverState`.

---

2. Client-side Design Doc

The client side is the visual/input/audio layer. It uses Slick2D for the game window and Swing for dialogs.

Purpose

The client side displays the game, handles mouse/keyboard input, plays audio, shows dialogs, and gives visual feedback to the user.

Main Responsibilities

- Open game window.
- Render background, players, cards, draw pile, discard pile.
- Handle mouse clicks on human cards.
- Handle UNO button click.
- Display player-name prompt.
- Display wild-card color selection dialog.
- Play sound effects and music.
- Display game-over screen.
- Apply UNO card icon to windows/dialogs.

Main Classes

`Game`

- Application entry point.
- Loads config.
- Loads audio.
- Creates Slick2D `AppGameContainer`.
- Sets display mode.
- Sets window icon.
- Starts state-based game.
- Registers:
  - `GameState`
  - `GameOverState`

`GameState`

- Main gameplay screen.
- Creates `UnoGame`.
- Starts the game loop on a separate thread.
- Every frame:
  - calls `game.update(container)`
  - calls `game.render(g)`
  - draws UNO button
  - draws UNO-call indicators

`GameOverState`

- Shows winner message.
- Shows restart/quit prompt.
- Input:
  - `Enter` or `O`: play again.
  - `Escape` or `N`: quit.

`Sprite`

- Loads all card PNG images.
- Provides image lookup methods for cards, card backs, overlays, and backgrounds.

`Audio`

- Loads WAV sound files and background music.
- Plays click, invalid click, UNO, win, and background music sounds.

`ColorSelectionDialog`

- Swing dialog with four buttons:
  - Blue
  - Yellow
  - Red
  - Green

- Used when human plays Wild or +4.

`ClientConfigWindow`

- Optional Swing config editor.
- Lets user change:
  - window width
  - window height
  - play music
  - play sounds

`AppIcon`

- Shared helper for app/window/dialog icon.
- Uses `res/gfx/hidden.png`.

Rendering Requirements

The background color depends on the discard pile top card:

- Blue: `6699FF`
- Green: `66FF99`
- Red: `FF6666`
- Yellow: `FFFF99`
- Black: `333333`

Cards:

- Human cards are shown face-up.
- Bot cards are shown as `hidden.png`.
- Unplayable human cards have `inactiveCard.png` overlay.
- Cards are fanned based on player seat.

Player positions:

- 2 players: bottom, top
- 3 players: bottom, right, left
- 4 players: bottom, right, top, left

UNO button:

- Bottom-right corner.
- Size: `120x40`.
- Text: `UNO`.
- Enabled only when current player has exactly 2 cards.

UNO indicators:

- Small circles near top-left.
- One per player.
- Green means UNO called.
- Red means not called.

Input Requirements

Human card click:

- Only current human player can click cards.
- If clicked card is playable:
  - set it as `playedCard`
  - release turn latch
  - play click sound
- If clicked card is not playable:
  - play invalid sound

UNO button click:

- If current player has 2 cards:
  - mark UNO called.
- Otherwise:
  - play invalid sound.

Game-over input:

- `Enter` / `O`: restart.
- `Escape` / `N`: quit.

Dialogs

Player name dialog:

- Prompt: `Player name?`
- Title: `UNO`
- Icon: `hidden.png`
- Empty/cancel result defaults to `Player`.

Color dialog:

- Title: `Choose a color`
- Icon: `hidden.png`
- User chooses color after playing Wild/+4.

---

3. Server-side Code

Server-side/game-engine files:

```text
src/game/Card.java
src/game/NumberCard.java
src/game/ActionCard.java
src/game/CardColor.java
src/game/CardSymbol.java
src/game/CardPile.java
src/game/DrawPile.java
src/game/DiscardPile.java
src/game/Hand.java
src/game/Player.java
src/game/HumanPlayer.java
src/game/Bot.java
src/game/SeatPosition.java
src/game/UnoGame.java
```

These files contain:

- Card definitions.
- Deck construction.
- Draw/discard pile behavior.
- Player behavior.
- Bot AI.
- Turn order.
- UNO rules.
- Win detection.
- Action-card effects.

Important engine entry point:

```java
UnoGame game = new UnoGame();
game.start(stateGame);
```

---

4. Client-side Code

Client-side/rendering/input/audio/dialog files:

```text
src/Game.java
src/GameRunnable.java
src/AppIcon.java
src/states/GameState.java
src/states/GameOverState.java
src/states/MenuState.java
src/gfx/Sprite.java
src/io/Audio.java
src/io/ColorSelectionDialog.java
src/io/ColorSelectionButtonActionListener.java
src/ClientConfigWindow.java
src/common/Config.java
src/common/Debug.java
```

These files contain:

- Main application startup.
- Slick2D window creation.
- Game states.
- Rendering.
- Mouse/keyboard input.
- Sprite loading.
- Audio playback.
- Swing dialogs.
- Config loading.
- App icon handling.

Important client entry point:

```java
public static void main(String[] args) throws SlickException
```

inside `Game.java`.

Build command:

```sh
./compile.sh
```

Run command:

```sh
./run.sh
```